

# ECEN 142L Communications and Networking Lab

## Lab 4: Convolutional Codes

Dylan Thornburg & Zach Anderson

Convolutional coding is a very powerful way for our communication systems to be resilient to errors. We have seen some examples of convolutional coding in lecture, but today we will dive a little deeper by building our own encoder and decoder from scratch.

Today's Objective: Gain some experience using encoding and decoding messages using standard techniques in convolutional coding.

### Part 1: Building out an Encoder

The first step we will do today is to build an encoder. To keep things relatively simple, we will use a constraint length of 3 (the length of the window in samples) and we will stick with the standard generator polynomials, namely:

$$P_0[n] = x[n] + x[n-1] + x[n-2] \quad P_1[n] = x[n] + x[n-1]$$

You may see this configuration commonly referred to as  $k = 3$  (7,6) where  $k$  is the constraint length, and 7 and 6 (111 and 110 respectively) are the coefficients applied.

These equations can be mapped to a physical implementation, as in the block diagram below:

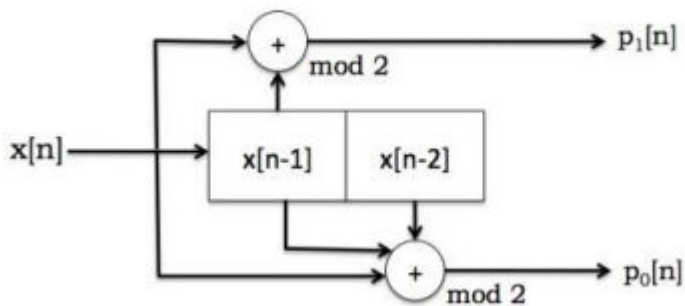
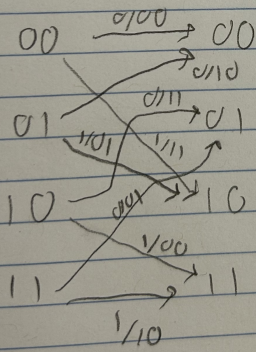
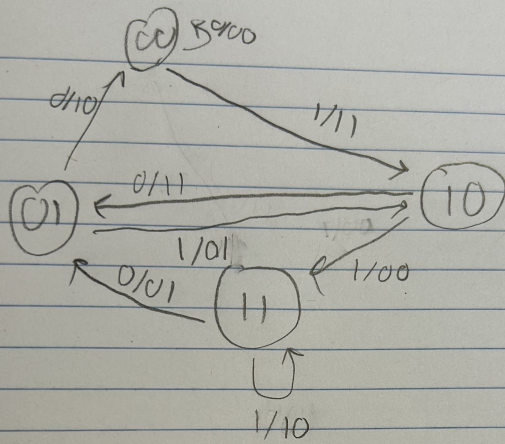


Figure 1: Convolutional encoder block diagram

Step 1: On a piece of paper, draw out the state diagram for this encoder. Label each state transition with  $X/YY$  where  $X$  is the input bit and  $YY$  is two output bits. Show your state diagram to the TA before you proceed.

PIPO = YY



yes completed  $\rightarrow$  [0111]

Lab 4 Bigrams

|       | $x[n]$ | $x[n-1]$ | $x[n-2]$ | $(x[n] + x[n-1] + x[n-2]) \% 2$<br>$PO[n]$ | $(x[n] + x[n-1]) \% 2$<br>$PI[n]$ |
|-------|--------|----------|----------|--------------------------------------------|-----------------------------------|
|       | 0      | x        | x        | x                                          | x                                 |
|       | 0      | 0        | x        | x                                          |                                   |
|       | 0      | 0        | 0        | 0                                          | 0                                 |
|       | 0      | 0        | 0        | 0                                          | 0                                 |
|       | 1      | 0        | 0        | 1                                          | 1                                 |
| 01011 | 1      | 1        | 0        | 0                                          | 0                                 |
| or    | 1      | 1        | 1        | 0                                          | 1                                 |
|       | 1      | 1        | 1        | 0                                          | 1                                 |
|       | 0      | 1        | 1        | 1                                          | 0                                 |
|       | 0      | 0        | 1        | 0                                          | 1                                 |
|       | 0      | 0        | 0        | 0                                          | 0                                 |
|       | 0      | 0        | 0        | 0                                          | 0                                 |
|       | 1      | 0        | 0        | 1                                          | 1                                 |
|       | 0      | 1        | 0        | 1                                          | 1                                 |
|       | 1      | 0        | 1        | 1                                          | 0                                 |
|       | 1      | 1        | 0        | 0                                          | 0                                 |
|       | 0      | 1        | 1        | 1                                          | 0                                 |
|       | 0      | 0        | 1        | 0                                          | 1                                 |

Step 2: Below is the function prototype for the encoder we will be creating. Open a new file, and paste the definition into the editor:

```
function [encoded] = ConvEncode(msgBits)
```

Your function will take msgBits, a 1-D row vector of binary digits as an input. For example:

```
msgBits = [1,0,1,1,0,0];
```

Your function should return another 1-D row vector as its output. Remembering that we start in state 00 when encoding, the correct output for the input above would look like:

```
encoded = [1,1,1,1,0,1,0,0,0,1,1,0];
```

Finish writing the ConvEncode function, using the above as a test case.

```
clear;
clc;
```

```
msgBits = [1,0,1,1,0,0];
encoded = ConvEncode(msgBits);
disp(encoded);
```

1 1 1 1 0 1 0 0 0 1 1 0

Step 3: Using your state diagram, compute the output for the message:

[0,1,0,1,1,1,0,0,1,0,1,0,0,0,1]

Show your results from Part 1 to the TA.

```
clear;
clc;

msgBits = [0,1,0,1,1,1,0,0,1,0,1,0,0,0,1];
encoded = ConvEncode(msgBits);
disp(encoded);
```

0 0 1 1 1 1 0 1 0 0 1 0 0 1 1 0 1 1 1

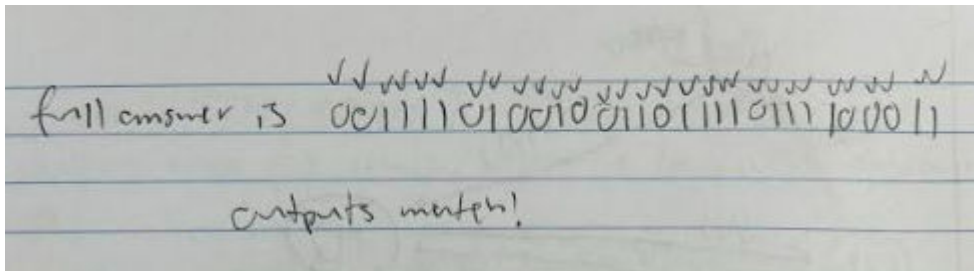
Now try the message using your function. Do the outputs match?

**Yes they do!**

output is P/P1

| input | curr state | next state | output | input | curr | next | out |
|-------|------------|------------|--------|-------|------|------|-----|
| 0     | 00         | 00         | 00     | 1     | 00   | 10   | 11  |
| 1     | 00         | 10         | 11     | 0     | 10   | 01   | 11  |
| 0     | 10         | 01         | 11     | 1     | 01   | 10   | 01  |
| 1     | 01         | 10         | 01     | 0     | 10   | 01   | 11  |
| 1     | 10         | 11         | 00     | 0     | 01   | 00   | 10  |
| 1     | 11         | 11         | 10     | 0     | 00   | 00   | 00  |
| 0     | 11         | 01         | 01     | 1     | 00   | 10   | 11  |
| 0     | 01         | 00         | 10     |       |      |      |     |





## Part 2: Helper Functions

Before we move on to decoding, we need to define a few helper functions to make our lives easier when we build the decoder.

Step 1: The first function we need is the hamming distance, which will be our metric for determining the most likely path. The hamming distance is simply the number of bit flips to go from one binary message to another. For example:

0101 0100 Need to flip one bit, Hamming Distance = 1

0101 **1**011 Need to flip three bits, Hamming Distance = 3

Note that the order doesn't matter, we could just as easily flip bits in either of the top binary sequences and come to the same conclusion. We should also note that we can never get a hamming distance greater than the number of bits in the number, ie:

0000 **1111** Need to flip all 4 bits, HD = 4. Impossible to get HD > 4.

Write a function HammingDistance(x, y) that returns the hamming distance between two binary arrays of the same length. Use the 3 examples above as test cases for your function.

```
clear;
clc;

x1 = [0 1 0 1];
y1 = [0 1 0 0];
HD1 = HammingDistance(x1,y1);

x2 = [0 1 0 1];
y2 = [1 0 1 1];
HD2 = HammingDistance(x2,y2);

x3 = [0 0 0 0];
y3 = [1 1 1 1];
HD3 = HammingDistance(x3,y3);

disp([HD1 HD2 HD3]);
```

1      3      4

Step 2: The second function we will create is a BER (Bit Error Rate) function. After all, the purpose of using a convolutional code is to prevent bit errors from corrupting the data. It will be useful to have a measure of how well our code works - BER will do this for us.

$BER = \text{num errors} / \text{num bits}$

Write a function ComputeBER(refBits, actualBits) that returns the calculated BER given the actual and received message. Show your results from Part 2 to the TA.

```
clear;
clc;

%testing seq 1
refBits = [0, 1, 0, 1];
actualBits = [0, 1, 0, 0];
BER = ComputeBER(refBits,actualBits);

disp(BER);
```

0.2500

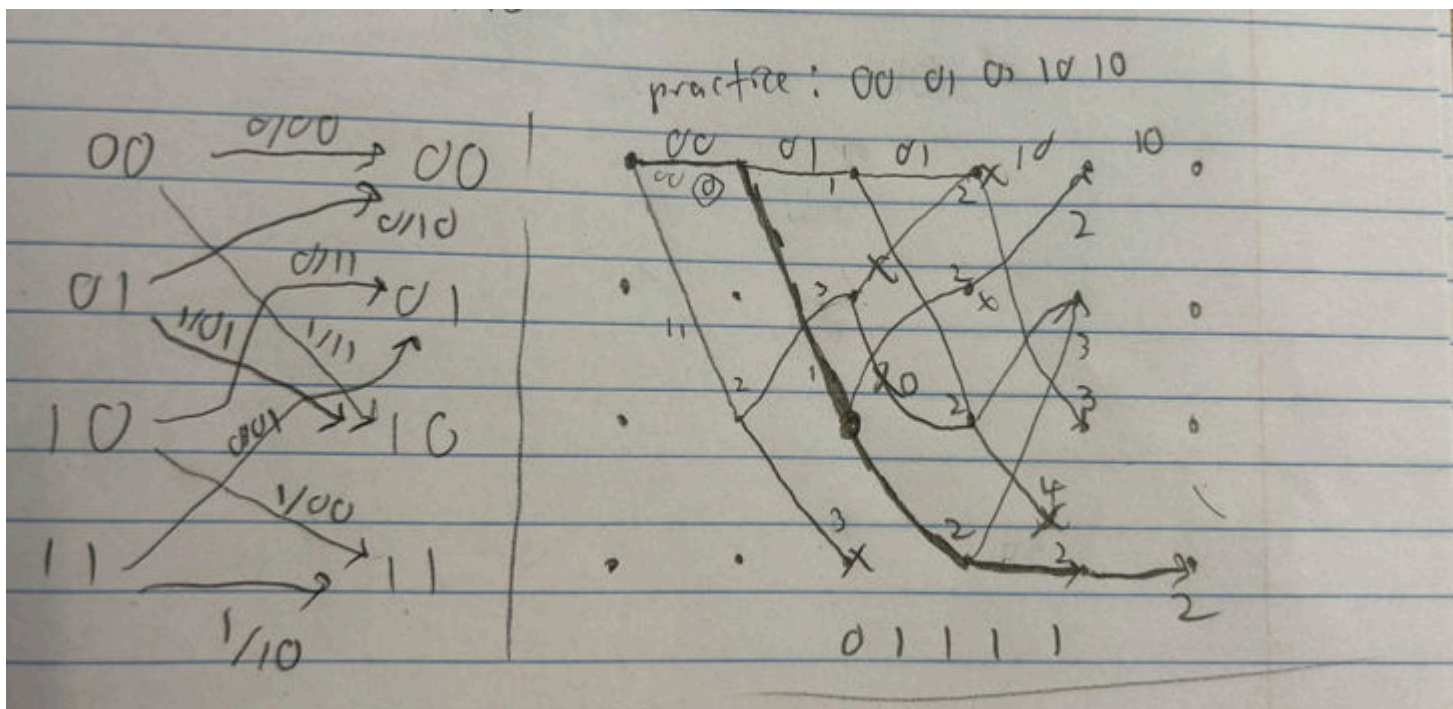
### Part 3: Building out a Decoder

With our encoder and some helper functions ready, we can now move on and create our decoder. Unfortunately, decoding isn't as quite as simple as encoding - we need to make an informed guess about what the encoder did from bits that were potentially corrupted by transmission. The Viterbi algorithm is the best solution to these problems, and while creating our own viterbi decoder is outside the scope of this class, we will learn about how it works and use MATLAB's implementation to decode some messages.

Step 1: Do a trellis diagram Before we start using MATLAB's decoder, let's take a minute to see how it works. The basic idea is fairly simple: using our knowledge of the encoder state machine and the input bits to the encoder, we can draw out all of the paths the encoder might have taken. Using our hamming distance metric, we can see then decide which one was the most likely.

Follow along with this video as it walks through how to create a trellis diagram: Digital Communications: Viterbi Algorithm. On a piece of paper, draw out the trellis and decode the message corresponding to the received sequence [00, 01, 01, 10, 10]. Show your work to the TA when you finish.

**Insert or summarize your trellis diagram and decoded message here after completing it on paper.**



Decoded message is 01111 (on bottom).

Step 2: For us to use the built-in Viterbi decoder, we need to give the MATLAB function some information about how our state machine works. Input the following code:

```
clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;
```

These lines tell the decoder that we have only 1 bit input (2 outcomes, 0 and 1), that for each input bit we can get 2 output bits (2 bits means 4 output symbols) and that our state machine has 4 states.

Next, we need to input how our state machine works. Fill out the following two tables:

Table 1: Next states

| State     | Bit inputs | Bit inputs |
|-----------|------------|------------|
|           | <b>0</b>   | <b>1</b>   |
| <b>00</b> | 00         | 10         |
| <b>01</b> | 00         | 10         |
| <b>10</b> | 01         | 11         |
| <b>11</b> | 01         | 11         |

Table 2: State outputs

| State     | Bit inputs | Bit inputs |
|-----------|------------|------------|
|           | <b>0</b>   | <b>1</b>   |
| <b>00</b> | 00         | 11         |
| <b>01</b> | 10         | 01         |
| <b>10</b> | 11         | 00         |
| <b>11</b> | 01         | 10         |

Now we can put these into the myTrellis structure. Enter your tables into matlab variables called next\_states and state\_outputs, and then paste the following code:

```
myTrellis.nextStates = next_states;
```

```
myTrellis.outputs = state_outputs;
```

```
istrellis(myTrellis)
```

Make sure the output is 1, and if it isn't, go back and fix any mistakes.

```
next_states = [0 2;
               0 2
               1 3
               1 3];
state_outputs = [0 3;
                 2 1
                 3 0
                 1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;
istrellis(myTrellis)
```

```
ans = logical
     1
```

Step 3: Use the following line to decode your message:

```
decoded = vitdec(encoded, myTrellis, 1, 'trunc', 'hard');
```

```
clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;
next_states = [0 2;
               0 2
               1 3
               1 3];
```



```

state_outputs = [0 3;
                 2 1
                 3 0
                 1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;
istrellis(myTrellis);
myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;

msgBits = [0,1,0,1,1,1,0,0,1,0,1,0,0,0,1];
encoded = ConvEncode(msgBits);
decoded = vitdec(encoded, myTrellis, 1, 'trunc', 'hard');

disp(msgBits);

```

0 1 0 1 1 1 0 0 1 0 1 0 0 0 1

```
disp(encoded);
```

0 0 1 1 1 1 0 1 0 0 1 0 0 1 1 0 1 1 1

```
disp(decoded);
```

0 1 0 1 1 1 0 0 1 0 1 0 0 0 1

Does your decoded message match the original?

**Yes it matches!**

Test out your encoder/decoder for all the message pairs presented so far. Show your results from Part 3 to the TA.

```

clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;
next_states = [0 2;
               0 2
               1 3
               1 3];
state_outputs = [0 3;
                 2 1
                 3 0
                 1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;
istrellis(myTrellis);
myTrellis.nextStates = next_states;

```

```

myTrellis.outputs = state_outputs;

msgBits = [1,0,1,1,0,0];
encoded = ConvEncode(msgBits);
decoded = vitdec(encoded, myTrellis, 1, 'trunc', 'hard');

disp(msgBits);

```

```

1    0    1    1    0    0

```

```

disp(encoded);

```

```

1    1    1    1    0    1    0    0    0    1    1    0

```

```

disp(decoded);

```

```

1    0    1    1    0    0

```

```

clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;
next_states = [0 2;
               0 2
               1 3
               1 3];
state_outputs = [0 3;
                 2 1
                 3 0
                 1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;
istrellis(myTrellis);
myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;

msgBits = [0 1 1 1 1];
encoded = ConvEncode(msgBits);
received = [0 0 0 1 0 1 1 0 1 0];
decoded = vitdec(received, myTrellis, 1, 'trunc', 'hard');

disp(msgBits);

```

```

0    1    1    1    1

```

```

disp(received);

```

```

0    0    0    1    0    1    1    0    1    0

```

```

disp(decoded);

```

0      1      1      1      1

## Part 4: Measuring Performance

So far, we have been simulating an ideal channel because no errors have occurred. However, if channels were really ideal, then we would have no need for convolutional codes! Lets see how our system does when we introduce some bit errors.

Step 1: First, let's define a way to create an error vector. Given a probability of error  $P = 0.02$ , write a code to output an error vector "errors" that is the same size as an encoded message that has random errors. Use the function rand: Uniformly distributed random numbers - MATLAB rand

(Hint: this can be done in 1 line of code)

```
clear;
clc;

msgBits = [0,1,0,1,1,1,0,0,1,0,1,0,0,0,1];
encoded = ConvEncode(msgBits);
P = 0.02;

errors = rand(size(encoded)) < P;
```

Step 2: Using the code below, corrupt the encoded message with our error vector, and compute the BER.

```
corrupted = mod(encoded+errors,2); BER_before = ComputeBER(encoded, corrupted);
```

Now run the decoder on the corrupted signal, and compute the BER on the decoded signal. Repeat this process multiple times until you observe a case where BER\_before is nonzero, whereas the BER on the decoded signal is zero. Comment on what this means.

```
clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;

next_states = [0 2;
0 2;
1 3;
1 3];

state_outputs = [0 3;
2 1;
3 0;
1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;
```



Step 3: Try a few different combinations of message lengths and probabilities of error and record your results. Is your convolutional code perfect? What could you do to make it better? Show your results from Part 4 to the TA.

### PROBABILITY OF ERROR = 50%

```
clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;

next_states = [0 2;
0 2;
1 3;
1 3];

state_outputs = [0 3;
2 1;
3 0;
1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;

istrellis(myTrellis);

msgBits = [0,1,0,1,1,1,0,0,1,0,1,0,0,0,1];
encoded = ConvEncode(msgBits);

P = 0.50;

BER_before = 0;
BER_after = 1;

while ~(BER_before ~= 0 && BER_after == 0)
    errors = rand(size(encoded)) < P;

    corrupted = mod(encoded + errors, 2);
    BER_before = ComputeBER(encoded, corrupted);

    decoded = vitdec(corrupted, myTrellis, 1, 'trunc', 'hard');
    BER_after = ComputeBER(msgBits, decoded);
end

disp(encoded);
```

0 0 1 1 1 1 0 1 0 0 1 0 0 1 1 0 1 1



```
disp(errors);
```

```
0 0 0 1 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1
```

```
disp(corrupted);
```

```
0 0 1 0 0 0 1 1 0 1 1 0 0 1 1 0 1 1 1 1
```

```
disp(decoded);
```

```
0 1 0 1 1 1 0 0 1 0 1 0 0 0 1
```

```
disp(BER_before);
```

```
0.3000
```

```
disp(BER_after);
```

```
0
```

**Even when probability is 50%, Trellis was still able to decode out all errors.**

## MESSAGE LENGTH IS 4 TIMES AS BIG.

```
clear;
clc;

myTrellis.numInputSymbols = 2;
myTrellis.numOutputSymbols = 4;
myTrellis.numStates = 4;

next_states = [0 2;
0 2;
1 3;
1 3];

state_outputs = [0 3;
2 1;
3 0;
1 2];

myTrellis.nextStates = next_states;
myTrellis.outputs = state_outputs;

istrellis(myTrellis);
```



Submit your results from Part 1 (Steps 1 and 3) and Part 3 (Steps 1 and 2), along with your codes and answers to questions on camino. In addition, discuss the following questions with your partner and attach your answers to the end of your report.

**1. What did you observe from the experiments?**

Trellis decoding is incredibly effective at innately correcting errors by virtue of how it's designed, and convolutional encoding is a great way to preserve information through noisy channels.

**2. If you could expand this experiment how would you do it?**

We personally feel that this lab covered pretty much everything. We both feel like we understand the need and process for encoding and decoding. Maybe try alternate encoding/decoding methods for future expansion.

**3. How does the experiment relate to the course topic?**

We have directly talked about encoding and decoding in lecture, so this lab is pretty much a 1:1 exercise of lecture 3.

**4. What did you learn from the experiment?**

Besides what we observed, we also learned how to draw Trellis diagrams and how the Viterbi algorithm works. We also got a refresher on state diagrams + in a networking context instead of logic design.

```
1 function [BER] = ComputeBER(refBits,actualBits)
2     if ~isequal(size(refBits), size(actualBits))
3         error('refBits and actualBits must have the same size.');
```

4 end

5

6 num\_errors = sum(refBits ~= actualBits);

7 num\_bits = length(refBits);

8

9 BER = num\_errors/num\_bits;

10 end

```
1 function [distance] = HammingDistance(x,y)
2     sum = 0;
3     for i = 1:length(x)
4         if x(i) ~= y(i)
5             sum = sum + 1;
6         end
7     end
8     distance = sum;
9 end
```

```
1 function [encoded] = ConvEncode(msgBits)
2     encoded = zeros(1, 2*length(msgBits));
3     x1=0;
4     x2=0;
5
6     for i = 1:length(msgBits)
7         x=msgBits(i);
8         p0 = mod(x+ x1 + x2, 2);
9         p1 = mod(x + x1, 2);
10
11         encoded(2*i -1) = p0;
12         encoded(2*i) = p1;
13
14         x2=x1;
15         x1=x;
16     end
17 end
```